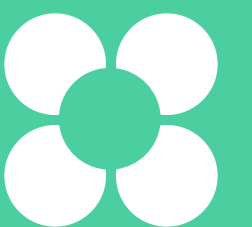


Функции в Go

Игорь Шамаев
Руководитель направления разработки



Игорь Шамаев

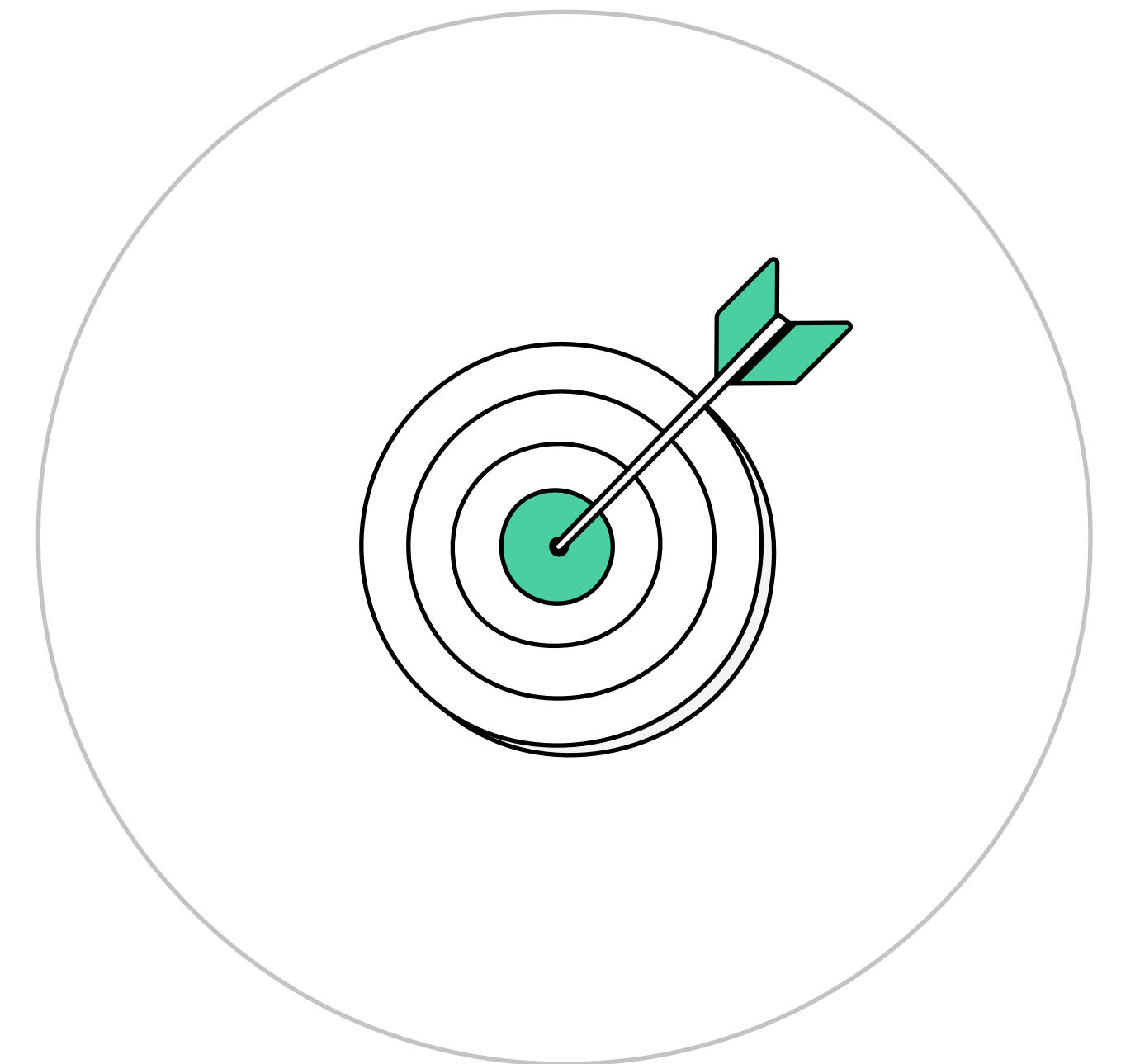
О спикере:

- Руководитель направления разработки в Домклик
- Развивает и поддерживает внутреннюю базу данных управления конфигурацией на Golang и TypeScript
- В прошлом — руководитель команды разработки в Сбере



Цели занятия

- Познакомить с понятием функции и их ролью в языке Go
- Освоить синтаксис объявления функций в Go
- Разобраться в сигнатуре функции и работе с параметрами и аргументами
- Ознакомиться с типичными ошибками при объявлении и вызове функций



План занятия

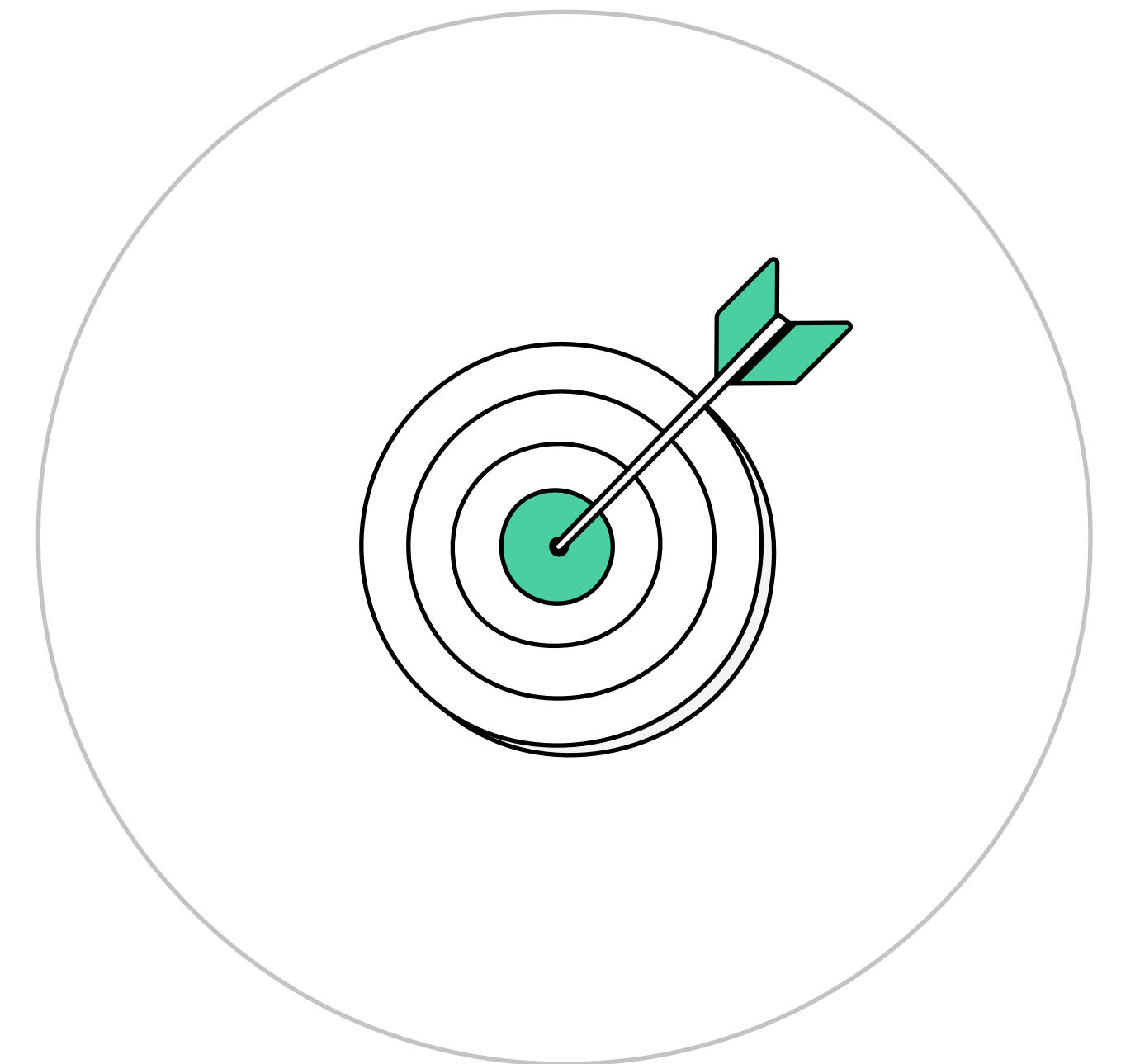
- 1 Принципы работы функций в языке Go
- 2 Как определять и вызывать функции в коде

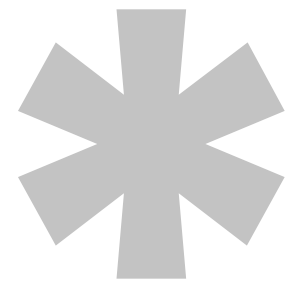


Принципы работы функций в языке Go

Цели видео

- Узнать, что такое функция в языке Go и зачем она нужна
- Разобраться, как объявлять функции в Go с помощью ключевого слова `func`
- Понять структуру объявления функции: имя, параметры, типы возвращаемых значений
- Рассмотреть, что такое сигнатура функции и из каких частей она состоит
- Узнать, почему функции важны для организации и повторного использования кода в Go





Функция — это именованный блок кода, выполняющий конкретную задачу

Она может принимать данные через параметры, обрабатывать их и возвращать результат. Функции предотвращают дублирование кода и упрощают его организацию

Введение в функции в Go

- В Go функции объявляются с помощью ключевого слова `func`
- Каждая программа на Go содержит функцию `main` - точку входа

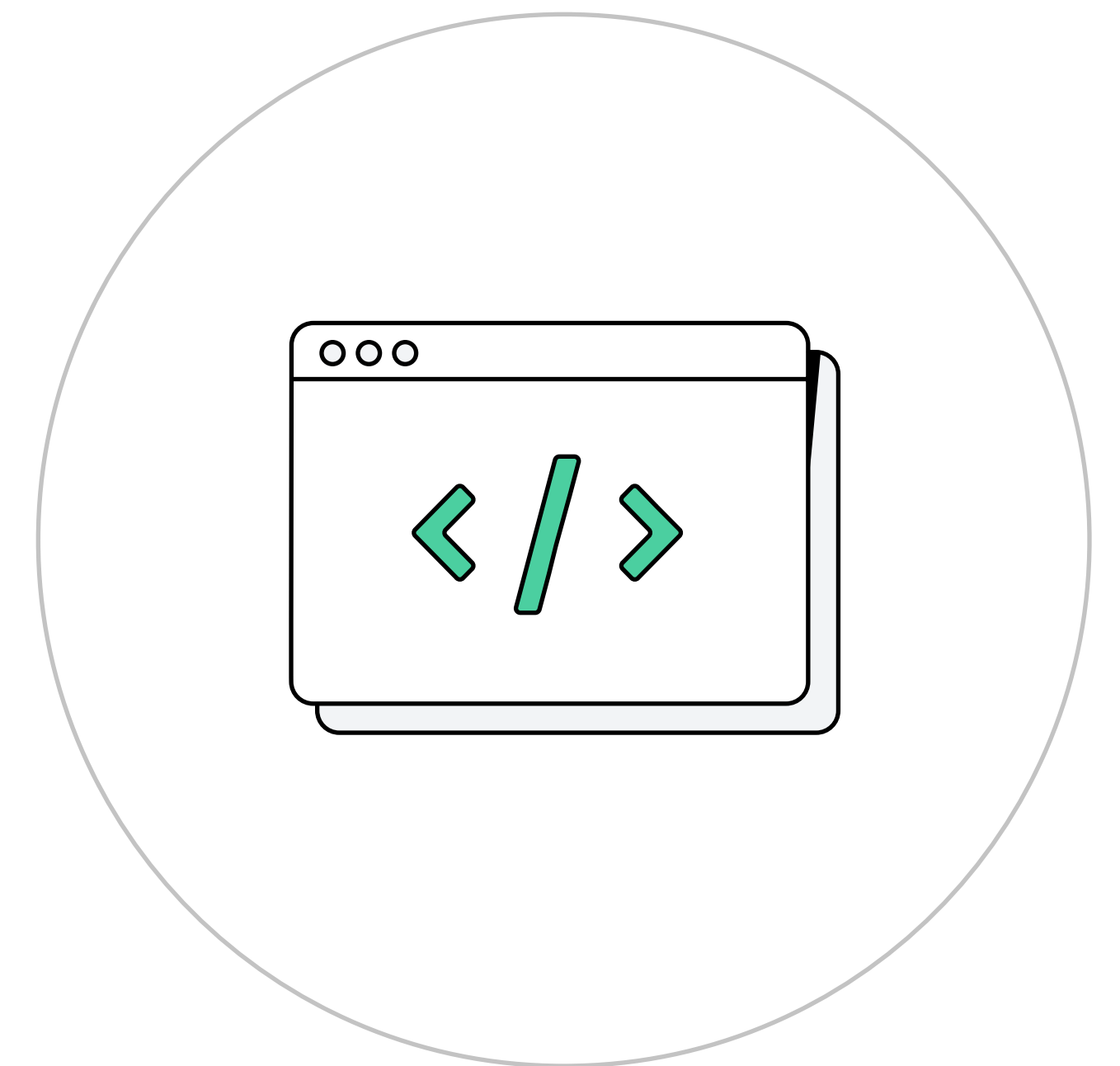
```
func main() {  
    fmt.Println("Hello Go")  
}
```


Функции как инструменты

Функции — это ваши личные помощники в коде

Как молоток в ящике с инструментами:

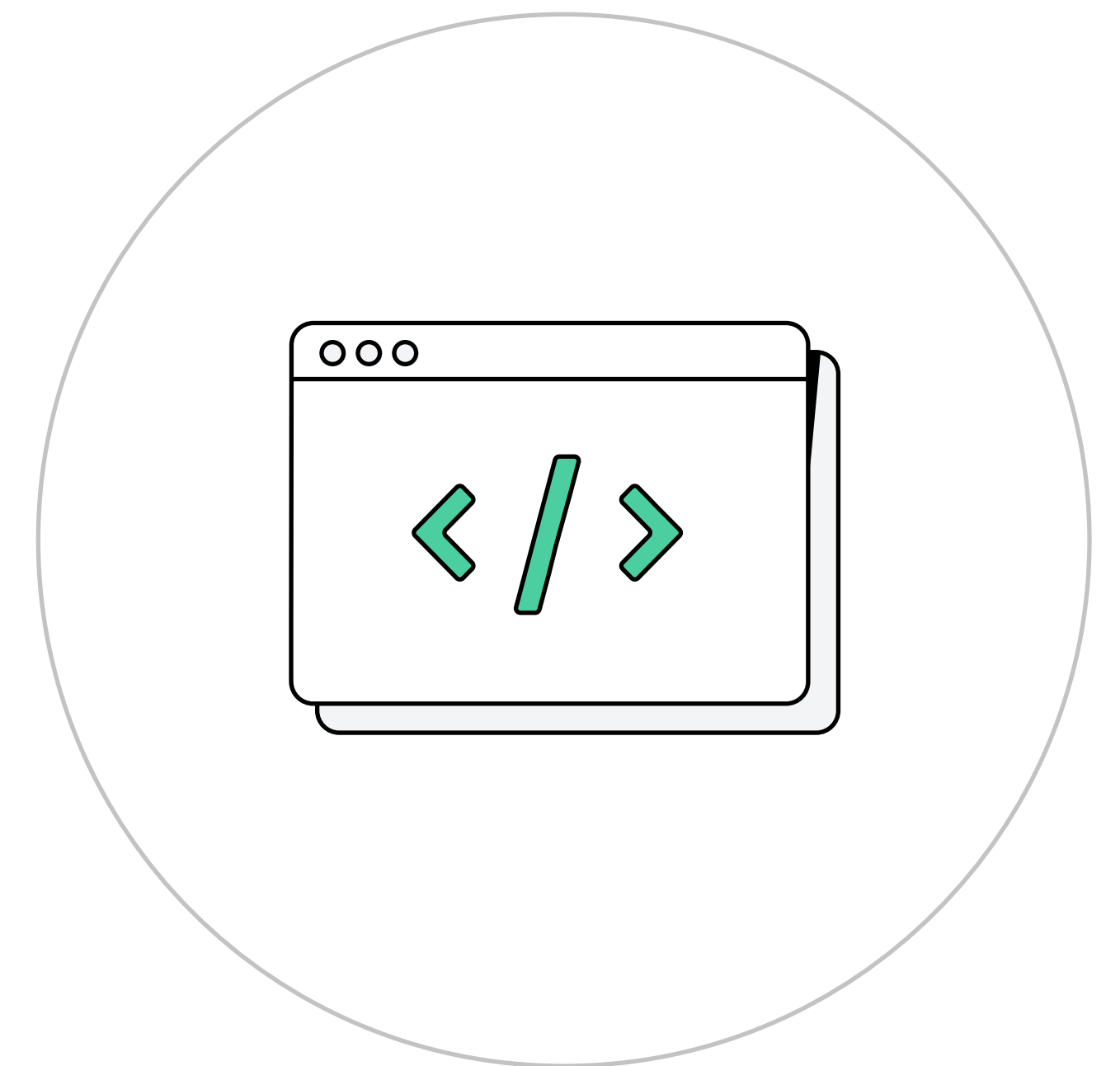
- Решают одну задачу
- Работают по вашему вызову
- Экономят время



Объявление функции

```
func имяФункции(параметры) типВозвращаемогоЗначения {  
    тело функции  
}
```

- func - ключевое слово
- Имя функции - camelCase, с большой буквы - экспортируемая функция
- Параметры указываются в скобках с типами
- Тип возвращаемого значения указывается после параметров (если есть)



Аналогия из жизни: функция — кассир

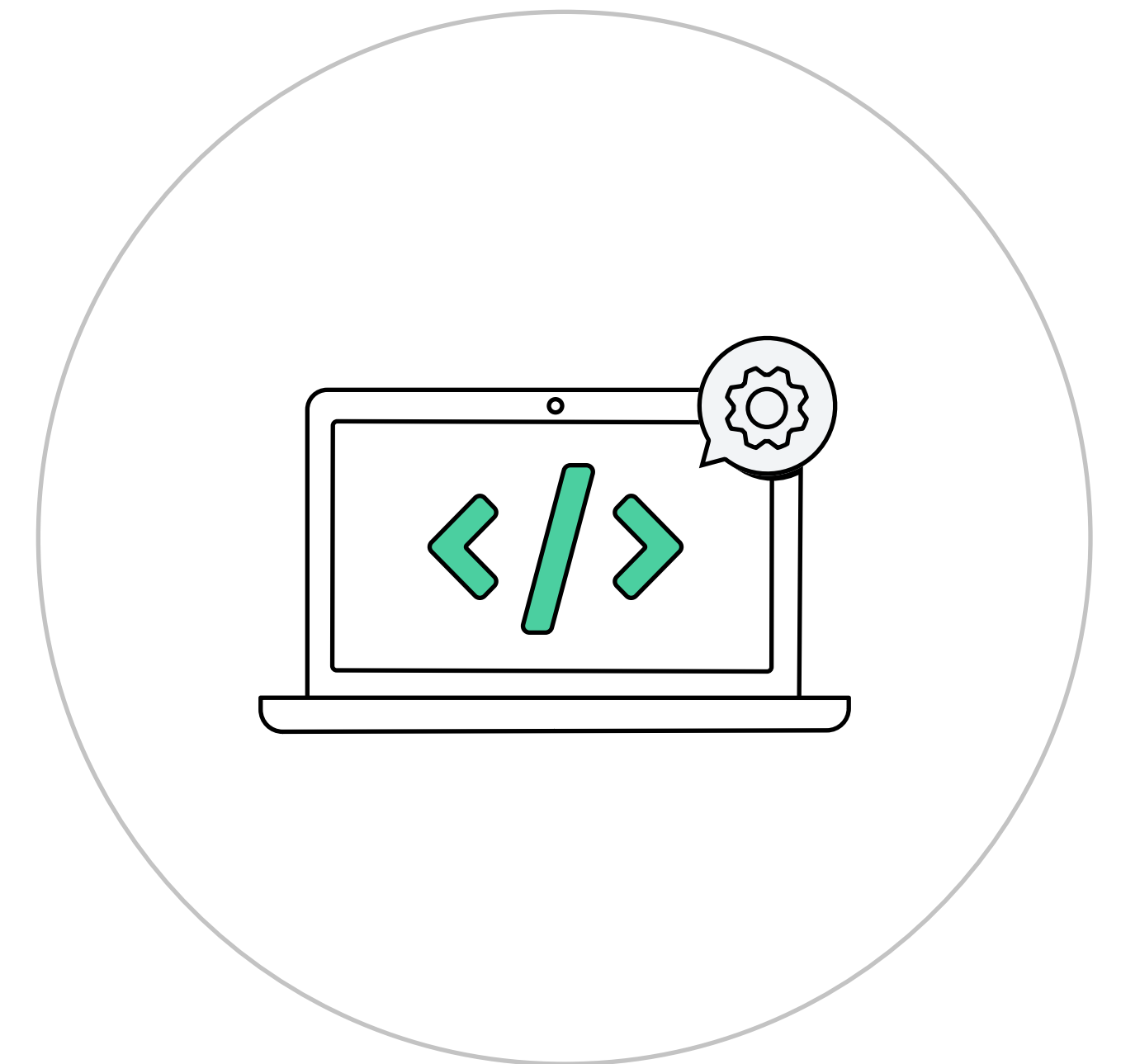
- 1 Вы передаёте товары (аргументы)
- 2 Кассир обрабатывает их (логика функции)
- 3 Возвращает итог (результат)

```
// Кассир, который складывает два числа
func add(a int, b int) int {
    return a + b
}

// Использование:
total := add(3, 5) // 8
```

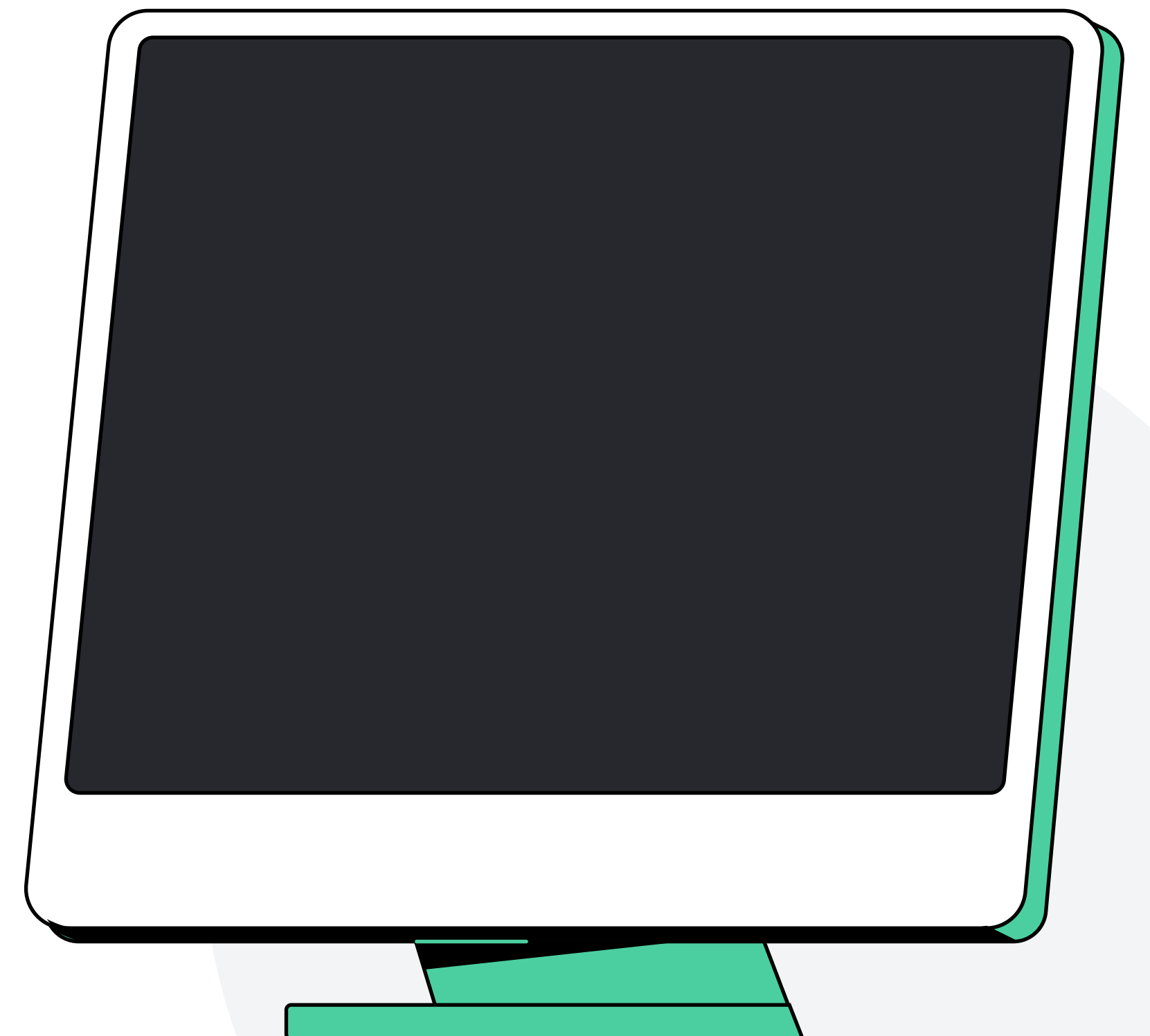

Параметры функции

- Можно указывать несколько параметров с одинаковым типом сокращённо: (x, y int)
- Функция может не иметь параметров:
`func sayHello() { ... }`
- Поддерживается переменное число параметров (вариадические функции):
`func sum(numbers ...int) int {...}`



Вызов функций и передача аргументов

- Вызов функции осуществляется по её имени с указанием аргументов в скобках
- Аргументы должны соответствовать типам параметров в сигнатуре функции
- Если функция возвращает значения, их можно присвоить переменным



Вызов функции: «Нажми на кнопку!»

```
result := greet("Анна")  
fmt.Println(result) // Привет, Анна
```

Вызов функции — это как нажатие на кнопку. Вы говорите: «Эй, greet, это "Анна" — поздоровайся!». Результат можно сохранить в переменную или сразу использовать

```
fmt.Println(greet("Иван")) // Привет, Иван
```


Сигнатура функции в Go

- Это её имя, список параметров и список возвращаемых значений
- Определяет, как функцию можно вызывать и какие данные она принимает и возвращает

```
func имя(параметры) (возвращаемые_значения)
```

Сигнатура функции — «Инструкция для кассира»

```
func calculateDiscount(price float64) float64
```

Она говорит:

- Как вас зовут? calculateDiscount
- Что вы принимаете? price типа float64
- Что возвращаете? float64

Почему функции важны в Go?

1

Порядок:

Функции делают код структурированным

2

Тестирование:

Легче проверить отдельный блок

3

Повторное использование:

Один раз написал — везде работает.
Принцип в DRY (Don't Repeat Yourself)



Итоги видео

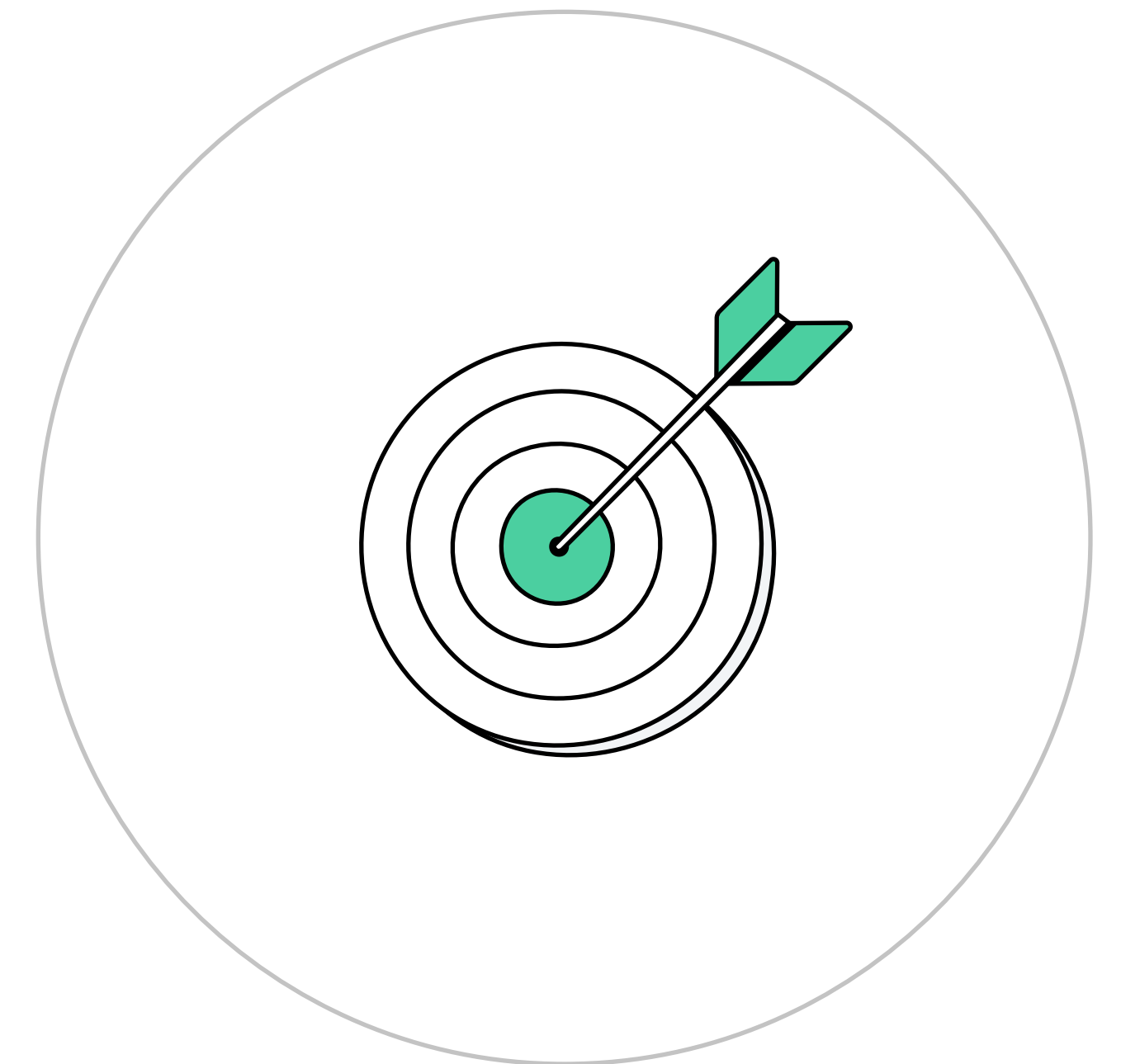
- Узнали, как объявлять функции в Go
- Поняли их важную роль в организации кода
- Рассмотрели структуру сигнатуры функции



Как определять и вызывать функции в коде

Цели видео

- Разобраться, как собирается функция по шагам
- Понять разницу между параметрами и аргументами функции
- Рассмотреть пример простой функции и функции с параметрами
- Узнать, как функция возвращает значения и как объявлять возвращаемый тип
- Понять типичные ошибки при объявлении и вызове функций



Синтаксис: собираем функцию по шагам

- 1 Ключевое слово `func``
- 2 Имя. Например, `greet``
- 3 Параметры в скобках `name string``
- 4 Тип возврата `string``
- 5 Тело функции `return ...``

```
func greet(name string) string {  
    return "Привет, " + name  
}
```

Параметры vs аргументы функции

Параметры (переменные в объявлении) — это сумма в рублях, которую кассир требует для оплаты.

Аргументы (значения при вызове) — купюры и монеты, которые вы кладёте в кассу

- Когда вы объявляете функцию, вы говорите:
«Я буду принимать параметр `name` типа `string`»
- Когда вызываете, передаёте аргумент: `greet("Анна")`
- Go строго проверяет типы: если вместо строки дать число, будет ошибка. Это как пытаться заплатить за хлеб камнем — кассир не поймёт

Простейшая функция

```
// Объявление функции
func sayHello() {
    fmt.Println("Привет, мир!")
}
func main() {
    sayHello() // Вызов функции → "Привет, мир!"
    sayHello() // Можно вызывать много раз
}
```

Функции с параметрами

```
// Функция принимает имя и выводит приветствие
func greet(name string) {
    fmt.Println("Привет,", name)
}

func main() {
    greet("Анна") // Привет, Анна
    greet("Иван") // Привет, Иван
}
```


Возвращаемые значения

```
// Функция складывает два числа и возвращает сумму
func add(a int, b int) int {
    result := a + b
    return result // return – ключевое слово для возврата
}

func main() {
    sum := add(3, 5) // Результат сохраняется в sum
    fmt.Println("Сумма:", sum) // Сумма: 8
}
```

Именованные возвращаемые значения

```
// sum и diff – именованные возвращаемые значения
func calc(a, b int) (sum int, diff int) {
    sum = a + b    // Присваиваем значение sum
    diff = a - b   // Присваиваем значение diff
    return         // Возвращает sum и diff автоматически
}

func main() {
    s, d := calc(10, 5)
    // Сумма: 15 Разность: 5
    fmt.Println("Сумма:", s, "Разность:", d)
}
```

Как объявить функцию?

```
func имяФункции(параметр1 тип, параметр2 тип) возвращаемыйТип {  
    // Логика...  
    return результат // если есть возвращаемое значение  
}
```

- Имя функции пишется с маленькой буквы (если не экспортируемая)
- Если функция не возвращает ничего, возвращаемый тип не указывается

```
func multiply(x int, y int) int {  
    return x * y  
}
```


Вызов функции

- 1 Пишете имя функции
- 2 Передаёте нужные параметры в скобках
- 3 Если функция возвращает значение — сохраняете его в переменную

```
func main() {  
    // Вызов функции с параметрами  
    greet("Мария")  
    // Вызов функции и сохранение результата  
    total := multiply(4, 5)  
    fmt.Println("Умножение:", total) // Умножение: 20  
}
```

Безымянные (анонимные) функции в Go

```
// Объявление и вызов
func() {
    fmt.Println("Я безымянная функция!")
}() // Вызов с помощью ()

// Сохранение в переменную
sumFunc := func(a, b int) int {
    return a + b
}

fmt.Println(sumFunc(3, 5)) // 8
```

Замыкания

Это функция, которая «захватывает» и сохраняет доступ к переменным из внешней области видимости, позволяя использовать и изменять их даже после завершения выполнения этой внешней функции

```
// Замыкание: анонимная функция захватывает внешнюю переменную
factor := 2 // внешняя переменная, к которой будет доступ
multiply := func(x int) int { // объявляем анонимную функцию
    return x * factor // используем «захваченную» переменную
}
fmt.Println(multiply(4)) // 8 выводит 4 * 2
fmt.Println(multiply(5)) // 10 выводит 5 * 2
```


Функция-генератор

```
// Функция-генератор: возвращает новую анонимную функцию
// функция, принимающая параметр a
func makeAdder(a int) func(int) int {
    return func(b int) int { // возвращает анонимную функцию
        return a + b // она складывает своё b и «захваченный» a
    }
}

add10 := makeAdder(10) // создаём функцию, «добавляющую» 10
fmt.Println(add10(7)) // 17 - выводит 10 + 7
```

Типичные ошибки

- Несоответствие типов

```
greetUser(10) // Ошибка! Ожидается string, передано число
```

- Лишние/недостающие параметры

```
add(3) // Ошибка! Ожидается два параметра
```

- Опечатка в имени функции

```
sayhello() // Ошибка! Правильно: sayHello()
```


Итоги видео

- Рассмотрели синтаксис функций в языке Go
- Узнали, как правильно создавать и вызывать функции
- Разобрались с типичными ошибками, которые могут возникать при работе



Итоги занятия

- Познакомились с понятием функции в языке Go
- Изучили, как правильно объявлять функции с помощью ключевого слова `func`, включая указание имени, параметров и типов возвращаемых значений
- Разобрались в структуре функции и её сигнатуре
- Рассмотрели типичные ошибки при объявлении и вызове функций



Функции в Go

Игорь Шамаев
Руководитель направления разработки

